



Trabajo Práctico Obligatorio: Seguridad, Autenticación y JWT

Desarrollar la siguiente práctica sobre el servidor API REST construido en el trabajo práctico anterior.

El objetivo del trabajo es incorporar un sistema de autenticación y autorización mediante JWT, agregando usuarios, registro, login, logout, rutas protegidas y favoritos asociados a usuarios autenticados.

No se debe crear una aplicación nueva. Cada grupo deberá trabajar sobre el servidor desarrollado en el TP de backend y sobre la aplicación React desarrollada en el TP 2.

La aplicación deberá permitir que distintos usuarios puedan registrarse, iniciar sesión y administrar sus favoritos dentro del sistema.

Los favoritos deberán persistir en la base de datos asociados al usuario autenticado, de manera que cada usuario conserve sus propios favoritos al cerrar sesión, volver a iniciar sesión o utilizar la aplicación desde otra sesión.

Requerimientos generales

El trabajo deberá incorporar:

- Registro de usuarios.
- Login de usuarios.
- Logout.
- Autenticación mediante JWT.
- Middleware de autenticación en el servidor.
- Hash de contraseñas antes de guardarlas en la base de datos.
- Entidad **User** en Prisma.
- Relación entre usuarios y favoritos.
- Endpoints protegidos para administrar favoritos.
- Formularios de registro y login en React.
- Integración del frontend con los endpoints de autenticación.
- Integración del frontend con los endpoints de favoritos.
- Persistencia de favoritos por usuario autenticado.
- Validaciones de datos de entrada.
- Respuestas con códigos HTTP adecuados.
- README actualizado en frontend y servidor.



Modelo de datos

Se deberá agregar una entidad **User** al schema de Prisma.

La entidad deberá incluir, como mínimo:

- id
- name o nombre
- email único
- password
- createdAt
- updatedAt

También se deberá agregar una relación entre User y la entidad principal del proyecto mediante una entidad de favoritos.

Cada favorito deberá pertenecer a un usuario y hacer referencia a un elemento existente del dominio de la aplicación.

No debe permitirse que un mismo usuario agregue dos veces el mismo elemento a favoritos.

Endpoints obligatorios del backend

Autenticación

Método	Endpoint	Descripción
POST	/auth/register	Registrar un nuevo usuario
POST	/auth/login	Iniciar sesión
POST	/auth/logout	Cerrar sesión
GET	/auth/me	Obtener los datos del usuario autenticado



Favoritos

Método	Endpoint	Descripción
GET	/favorites	Listar los favoritos del usuario autenticado
POST	/favorites/:id	Agregar un elemento a favoritos
DELETE	/favorites/:id	Quitar un elemento de favoritos

En los endpoints de favoritos, el usuario no debe recibirse por body ni por parámetro. Debe obtenerse a partir del token JWT enviado en la request.

Registro de usuarios

El endpoint de registro deberá:

- Validar los datos obligatorios.
- Verificar que no exista otro usuario con el mismo email.
- Hashear la contraseña antes de guardarla.
- Crear el usuario en la base de datos.
- No devolver la contraseña en la respuesta.

Códigos HTTP esperados:

- **201 Created** para registro correcto.
 - **400 Bad Request** cuando falten datos o sean inválidos.
 - **409 Conflict** cuando el email ya está registrado.
-

Login de usuarios

El endpoint de login deberá:

- Validar email y contraseña.
- Buscar el usuario por email.
- Comparar la contraseña recibida con la contraseña hasheada.
- Generar un token JWT si las credenciales son correctas.
- Devolver el token y los datos básicos del usuario.



Códigos HTTP esperados:

- **200 OK** para login correcto.
 - **400 Bad Request** cuando falten datos.
 - **401 Unauthorized** cuando las credenciales sean inválidas.
-

Logout

El endpoint de logout deberá existir en la API.

Para una implementación básica con JWT, puede responder correctamente y dejar que el frontend elimine el token almacenado.

En caso de implementar refresh token, el logout deberá invalidar, eliminar o revocar el refresh token correspondiente.

Favoritos en backend

Los favoritos deberán funcionar únicamente para usuarios autenticados.

Agregar favorito

El endpoint para agregar favoritos deberá:

- Requerir autenticación.
- Validar que el elemento exista.
- Validar que el elemento no haya sido agregado previamente por el mismo usuario.
- Crear la relación entre el usuario autenticado y el elemento favorito.

Códigos HTTP esperados:

- **201 Created** si el favorito se agrega correctamente.
- **401 Unauthorized** si no se envía token.
- **403 Forbidden** si el token es inválido.
- **404 Not Found** si el elemento con id no existe.
- **409 Conflict** si el favorito ya existe.



Quitar favorito

El endpoint para quitar favoritos deberá:

- Requerir autenticación.
- Validar que el favorito exista para el usuario autenticado.
- Eliminar la relación correspondiente.

Códigos HTTP esperados:

- **200 OK** si el favorito se elimina correctamente.
- **401 Unauthorized** si no se envía token.
- **403 Forbidden** si el token es inválido.
- **404 Not Found** si el favorito no existe para ese usuario.

Listar favoritos

El endpoint para listar favoritos deberá:

- Requerir autenticación.
- Devolver únicamente los favoritos del usuario autenticado.
- Incluir información del elemento favorito, no solamente su id.

Frontend

La aplicación React desarrollada en el TP 2 deberá integrarse con el sistema de autenticación del servidor.

Se deberán agregar, como mínimo:

- Página o componente de registro.
- Página o componente de login.
- Acción de logout.
- Manejo del usuario autenticado en la aplicación.
- Integración con los endpoints de favoritos.
- Visualización clara de favoritos del usuario.

La aplicación deberá enviar el token JWT en las requests protegidas utilizando el header Authorization.

Los favoritos no deben persistir en localStorage. Deben guardarse en la base de datos a través de la API y estar asociados al usuario autenticado.



Registro en frontend

La pantalla o formulario de registro deberá permitir crear un nuevo usuario utilizando el endpoint correspondiente del backend.

El formulario deberá contemplar, como mínimo:

- Nombre.
- Email.
- Contraseña.
- Mensajes de error ante datos inválidos.
- Mensaje o comportamiento claro cuando el registro sea exitoso.

Luego de registrarse, la aplicación podrá iniciar sesión automáticamente o redirigir al usuario a la pantalla de login.

Login en frontend

La pantalla o formulario de login deberá permitir iniciar sesión utilizando el endpoint correspondiente del backend.

El frontend deberá:

- Enviar email y contraseña al servidor.
- Recibir el token JWT.
- Guardar el token para mantener la sesión del usuario.
- Mantener disponible la información básica del usuario autenticado.
- Permitir acceder a las funcionalidades privadas de favoritos.

Cuando el usuario no esté autenticado, la aplicación no deberá permitir agregar o quitar favoritos.

El token puede ser guardado en localStorage o utilizar cookies.

Logout en frontend

La aplicación deberá permitir cerrar sesión.



Al cerrar sesión, el frontend deberá:

- Llamar al endpoint de logout.
 - Eliminar el token almacenado.
 - Limpiar la información del usuario autenticado.
 - Actualizar la interfaz para reflejar que no hay una sesión activa.
-

Seguridad mínima esperada

Se deberá cumplir con las siguientes condiciones:

- Las contraseñas no deben guardarse en texto plano.
 - El password no debe devolverse en ninguna respuesta.
 - El JWT debe firmarse utilizando una variable de entorno.
 - Las rutas privadas del servidor deben estar protegidas mediante middleware.
 - El usuario autenticado debe obtenerse desde el token.
 - No se debe confiar en ids de usuario enviados desde el cliente.
-

README

Se deberán actualizar los README correspondientes para agregar las nuevas funcionalidades y librerías.

Modalidad de entrega

Continuar trabajando sobre los mismos repositorios en los cuales los profesores ya tenemos acceso. Hacer pull request y commits sobre ese mismo repositorio.

No se requiere tablero Kanban para este trabajo práctico, aunque si lo necesitan para organizarse es bienvenido. Aun así, deberían seguir las mismas prácticas de pull request vistas en la materia.

Fecha límite para el último commit: **Lunes 29/06/2026 23:59**



Criterios de evaluación

Se evaluará:

- Correcta implementación de registro, login y logout.
- Correcto uso de JWT.
- Correcta implementación del middleware de autenticación.
- Protección de rutas privadas.
- Correcta creación de la entidad usuario.
- Correcta relación entre usuario y favoritos.
- Password hasheado en la base de datos.
- Validaciones de entrada.
- Uso adecuado de códigos HTTP.
- Integración entre frontend y backend.
- Formularios de registro y login funcionando.
- Manejo correcto de sesión en React.
- Persistencia de favoritos por usuario.
- Claridad de las respuestas de error.
- Organización del código.
- Documentación de ambos README.
- Funcionamiento general de la aplicación fullstack.

Bonus opcional: Refresh Token

Como funcionalidad adicional, se podrá implementar refresh token.

En ese caso, el login deberá devolver un access token y un refresh token. El access token deberá tener una duración corta, mientras que el refresh token permitirá obtener un nuevo access token sin volver a iniciar sesión.

La implementación del refresh token deberá contemplar:

- Persistencia del refresh token o mecanismo equivalente.
- Fecha de expiración.
- Endpoint para renovar access token.
- Invalidación del refresh token al hacer logout.
- Validación de refresh tokens vencidos o revocados.
- Integración del refresh token con el frontend.



Facultad de Informática
Programación Web Avanzada



Endpoint adicional esperado:

Método	Endpoint	Descripción
POST	/auth/refresh	Generar un nuevo access token usando un refresh token válido